

Felsökningsguide

Läsa en misslyckad workflow-körning

Kontinuerlig integration och leverans, L03

Syftet med guiden

Din pipeline kommer att bli röd, och det är normalt. Skillnaden mellan att fastna i en timme och att lösa det på fem minuter är oftast att man **läser loggen i rätt ordning** i stället för att gissa. Guiden visar hur du hittar felet, vilka sorters fel du kommer att möta, och hur du ber om hjälp så att någon kan hjälpa dig snabbt.

Du får öva på allt här i felsökningsverkstaden, med ett övningsrepo där pipelinen är trasig på flera ställen.

1. Hur en körning är uppbyggd

En **händelse** (till exempel en push) startar ett **workflow**. Workflowet har ett eller flera **jobb**, och varje jobb består av **steg** som körs ett efter ett på en **runner**, alltså en tillfällig dator som GitHub startar åt dig.

Det viktigaste att veta: **när ett steg misslyckas hoppas alla följande steg i jobbet över**. Därför ser du bara det första felet. När du har rättat det kan nästa fel dyka upp, och det är inte ett tecken på att du gjorde fel.

2. Läs en körning steg för steg

1. Hitta körningen. Öppna fliken **Actions** i ditt repo. Klicka på den senaste körningen. Ett rött kryss betyder misslyckad, en grön bock betyder lyckad, en gul prick betyder att den pågår.

2. Hitta jobbet. Klicka på jobbet till vänster (i vårt fall check).

3. Hitta det första röda steget. Stegen står i ordning. Det första med rött kryss är det som gick fel. Steg som är gråa har aldrig körts.

4. Läs loggen nerifrån. Öppna det röda steget. Det verkliga felmeddelandet står oftast **längst ned**, strax före raden med `Error: Process completed with exit code 1`. Den raden säger bara att något föll, inte vad.

5. Skriv ned felet med egna ord. Vilken fil? Vilken rad? Vad står det att programmet förväntade sig, och vad fick det?

6. Kör samma kommando lokalt. Stegets kommando står i `.github/workflows/ci.yml`. Kör det i din terminal. Får du samma fel går det mycket snabbare att prova rättningar än att pusha och vänta på en ny körning.

Tips: tryck på pilen vid ett steg för att öppna eller stänga det. Sökrutan i loggen kan hitta ordet *error* eller *failed*.

3. Tre platser där fel kan ligga

Fråga dig alltid först **var** felet finns. Det avgör hur du går vidare.

Plats	Typiskt tecken	Första åtgärd
Själva workflowfilen	Körningen startar inte, eller ett steg fallerar innan din kod ens körs	Läs filen noga: indrag, stavning, namn på actions
Miljön på runnern	Något "fungerar hos mig" men inte på GitHub: en fil, ett paket eller en version saknas	Fråga: finns det runnern behöver i repot på GitHub, inte bara på min dator?
Din egen kod eller dina tester	Ett kommando du känner igen (ruff, pytest) rapporterar ett fel i en fil	Kör kommandot lokalt och läs felet

4. Vanliga felmeddelanden och vad de betyder

Exemplen nedan är hämtade från samma sorts projekt som ni arbetar med, men de är inte identiska med felen i övningen. Du behöver alltså tänka själv när du möter dem.

Ogiltig workflowfil

```
Invalid workflow file: .github/workflows/ci.yml#L9
You have an error in your yaml syntax on line 9
```

YAML är känsligt för **indrag**. Alla steg i en lista ska ha exakt samma antal mellanslag före strecket. Räkna mellanslagen och jämför med raderna ovanför och nedanför. Använd aldrig tabbar. Texten ovan visar hur ett sådant fel *kan* se ut; exakt formulering kan skilja.

Fil som saknas

```
Error: ENOENT: no such file or directory, open 'requirements.txt'
```

Programmet letar efter en fil som inte finns på runnern. Fråga dig: finns filen i repot på GitHub? Kolla filiken Code. Ligger den i `.gitignore`, eller har du glömt att committa den?

Paket som saknas

```
ModuleNotFoundError: No module named 'pandas'
```

Koden importerar något som inte är installerat i miljön. Titta var paketet ska deklarerats (i det här kursprojektet i `pyproject.toml`) och hur det installeras. Att det fungerar på din dator betyder bara att du har paketet där.

Lint-fel (ruff check)

```
F841 Local variable `result` is assigned to but never used
--> src/app/model.py:12:5
```

Verktyget har hittat något i koden som det anser vara fel. Kodraden (`fil:rad:kolumn`) står i loggen. Läs regelkoden (här F841) och sök på den i ruffs dokumentation om du inte förstår. Ofta kan `ruff check --fix` rätta det åt dig.

Formateringsfel (ruff format --check)

```
Would reformat: tests/test_model.py
1 file would be reformatted
```

Det här är inget logikfel. Filen följer inte formatmallen. Kör `uv run ruff format` lokalt, titta på ändringen och committa.

Test som misslyckas (pytest)

```
FAILED tests/test_scale.py::test_scale_zero_to_one - assert 0.5 == 1.0
1 failed, 6 passed
```

Läs hur testet **väntade sig** att det skulle bli och vad det **fick**. Börja sedan i funktionen som testet anropar. Ändra inte testet för att få det grönt innan du är säker på att det är testet som har fel.

Föråldrad eller saknad låsfil

```
error: The lockfile at `uv.lock` needs to be updated, but `--locked` was provided.
```

Låsfilen beskriver exakt vilka versioner som ska installeras, och den ska följa med i repot. Har du ändrat beroenden? Kör `uv lock`, och committa både `pyproject.toml` och `uv.lock`.

5. Metoden: ett fel i taget

- **Ändra en sak åt gången.** Ändrar du tre saker och körningen blir grön vet du inte vilken som hjälpte, och du kan ha lagt in nya fel.
- **Förvänta dig fler fel.** När ett steg går grönt körs nästa, och det kan vara rätt av en helt annan orsak.
- **Gissa inte, läs.** Om du inte kan förklara med egna ord vad loggen säger, läs den igen innan du ändrar något.
- **Skriv upp vad du gjorde.** En rad per fel: vad stod i loggen, hur hittade du orsaken, hur löste du det. Det är det som gör att du kan lösa nästa liknande fel snabbare, och det är värdefullt material i din reflektion.
- **Kör lokalt när du kan.** Samma kommando som i workflowet, samma resultat, mycket snabbare svar.

6. Fastnat? Så ber du om hjälp

En bra hjälpfråga gör att någon kan svara på två minuter. Ta med:

- **Länk till körningen** (kopiera adressen från webbläsaren). Då kan den som hjälper dig se loggen själv.
- **Vilket steg som är rätt** och de sista 5–10 raderna av felet. Klistra in som text, inte som bild, om du kan.
- **Vad du förväntade dig** att skulle hända.
- **Vad du redan har provat.** Då slipper ni göra om samma sak.

Ett exempel: "Steget 'uv run pytest' är rätt i den här körningen: [länk]. Det står `ModuleNotFoundError` för `pandas`. Jag förväntade mig att det skulle installeras. Jag har kollat att det importeras i `model.py`. Jag vet inte var jag ska titta härnäst."

7. Checklista att ha vid sidan av

- Vilket är det **första** röda steget?
- Vad står det **längst ned** i det stegets logg?
- Kan jag **förklara felet med egna ord**?
- Är felet i **workflowfilen**, i **miljön** eller i **min kod**?

- Kan jag **köra samma kommando lokalt**?
- Har jag ändrat **en sak**, och noterat den?
- Skulle någon annan kunna hjälpa mig utifrån länk + felrad + vad jag provat?